

Table of contents

1

Value Proposition

2

Business Outcomes

3

Observability Framework Setup Steps

Play 1-4

4

Next Steps

Note: Every customer requirement and setup is unique. This playbook provides a general framework for how Datadog TAMs can help support customers as they implement their observability solution.

1

Value Proposition

This playbook provides a focused approach toward the specific decisions that must be made when setting up an observability framework from scratch or modernizing an existing organizational system. Each play aligns specific data collected and retained with its intended audience and purpose, highlighting areas where the business can capture value based on the guidance provided.

A large number of systems and services is not a barrier to adopting an effective observability framework. Conversely, greater complexity increases the need for a well-defined observability framework to prevent visibility gaps along business-critical verticals.

2

Business Outcomes

- Develop an observability framework that sustainably enables key business strategy
- Decrease time to realized value by standardizing and surfacing core metrics (**Play 1**)
- Improve availability and IT uptime with a proactive monitoring strategy (**Play 2**)
- Reduce operational costs for observability with a well-defined tagging structure and efficient FTE allocation for implementation (**Play 2**)
- Decrease MTTD/R and resources allocated to firefighting incidents with precise, effective alerts, resulting in reduced operational costs and protection against customer revenue loss (**Plays 3 & 4**)
- Manage risk with maintainable, continuous improvement processes (**Next Steps**)

3

Observability Framework Setup Steps

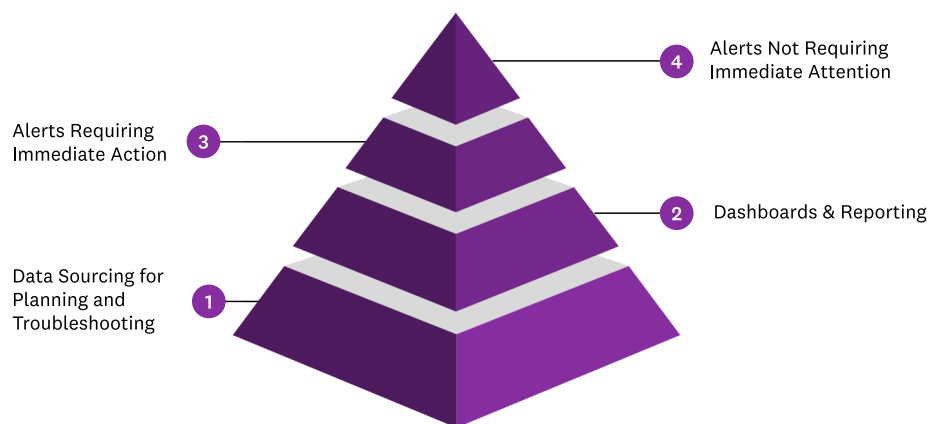
To create a robust and sound observability framework, it's important to create a hierarchy that meets both an organization's visualization and reporting needs. Each play in this playbook defines a level that creates a repeatable, actionable structure. The plays are organized by delivery of greatest value to the business and intended order of adoption.

Play 1 - Data Sourcing for Planning and Troubleshooting: **Creating Tagging Standards**

Play 2 - Dashboards and Reporting: **SLIs and SLOs**

Play 3 - Alerts Requiring Immediate Action

Play 4 - Alerts Not Requiring Immediate Attention



Play 1

Data Sourcing for Planning and Troubleshooting

Initially, [configure the Agent](#) to collect data, and complete cross-team validation to align expectations that the data collected is complete, correct, and scoped appropriately. Then install all core integrations with an eye toward value. Included metrics incur no additional cost and set a baseline for key product metrics, decreasing time to value. These metrics are standard and populate [OOTB dashboard](#) content within Datadog.

Note: If time-to-value is the core driver at this stage, consider using [live processes](#) to autode-
tect integrations running on your hosts and guarantee complete coverage.

Define additional metrics along with their associated business needs. The goal is to consider all possible data that may be useful when troubleshooting issues, including data that may not ever be used. The list below of possible metric categories is a starting point for the discussion to make sure there are no gaps on any individual product team.

Custom metrics will incur additional cost, but this can often be mitigated. In order to minimize the cost associated with storing and searching, configure custom metrics in as low a cardinality as possible (using a finite, bounded set of possible values) or choose a standard correlative metric to use instead. When defining custom metrics, associate them to a team to enable usage attribution and so the original purpose is not lost.

Sample list of metric categories in order of ease-of-implementation:

Infrastructure Metrics: CPU Usage / Memory Usage / Disk Space Utilization / Network Traffic (Inbound/Outbound) / Load Average

Containerization Metrics: Container CPU Usage / Container Memory Usage / Container Network Metrics / Container Restart Rate / Resource Limits and Requests

Web Server Metrics: HTTP Response Codes / Requests Per Second / Active Connections / Request+Response Sizes

Custom Application-Specific Metrics: Business Transactions / Key Performance Indicators (KPIs) / Custom Events and Triggers / Application-Specific Errors

User Experience Metrics: Page Load Time / User Session Duration / Click-through Rates / Conversion Rates / Bounce Rates

Application Performance Metrics: Response Time / Request Rate / Error Rate / Throughput / Latency

Database Metrics: Query Performance / Connection Pool Usage / Lock Waits / Index Usage / Transactions Per Second

Network Metrics: Network Latency / Packet Loss / Network Errors / Bandwidth Usage

Security Metrics: Failed Login Attempts / Unauthorized Access Attempts / Security Vulnerabilities / User Activity Logs

Key Benefits:

- Reduce time to access integration data on the platform through rapid setup and scaling
- Reduce consumption costs by optimizing search and storage requirements and minimizing unnecessary custom metrics

Create Tagging Standards

Tags are used to categorize and filter data so that teams are shown what is important to them.

Tags are the primary tool used to associate information with user groups, teams, and roles within an observability framework. They allow dashboards, reports, and notifications to be more granular, which enables engineers to filter data and find the root cause of issues more quickly, reducing MTTR.

While the observability framework for a specific application or service may be unique, tags should be considered from a global perspective to reduce overhead and simplify queries. Defining standards for tags and their expected values also reduces the learning curve for new users and ensures consistency among the apps your organization is monitoring.

Confirm that [unified service tagging](#) is in place across your infrastructure. Identify areas where these core tags are missing, and resolve this. This is a prerequisite, as inconsistent or incomplete tagging (for example, env:prod, env:prd, or env:N/A) can lead to the omission of critical data in visualizations scoped by environment, compromising the accuracy of reporting.

To enforce that these tags are in place, periodically review your configuration by using queries like the following to identify containers that are missing env, service, or version tags:

The screenshot shows the Datadog query editor interface. At the top, the title bar reads "Containers by env, service, version". Below the title bar, there's a "Table" view selector and a "Global Time" toggle. The main query area shows a query starting with "a Metrics" followed by a dropdown menu. The selected metric is "kubernetes.containers.running". The query is then filtered by "from !short_image:agent". Below this, the "sum by" clause lists "kube_container_name", "env", "service", "version", and "owner". The "reduce values in timeframe to" section is set to "avg" with a summation symbol. At the bottom, there are buttons for "+ Add Query" and "+ Add Formula".

Sample query to report on containers missing env, service, or version tag

Subsequently, discuss additional tags based on team and business unit. As workflows and audiences are defined for dashboards, establishing tags as early in the process as possible will help minimize potential rework.

Key Benefits

Provides the building blocks for capturing the key operational efficiency benefits of Datadog's unified observability platform, including:

- Dashboards providing visibility into system health and business performance
- Targeted alerting on critical components of your environments
- Reduced MTTR and FTEs during live incidents

With improved visibility, developers and engineers have more time to focus on core business activities such as developing new features and increasing customer conversion.

Play 2

Dashboards and Reporting

The data used in dashboards and standard reports should be a significantly smaller subset than the total scope of ingested and retained data.

In order to select data for dashboards and reporting, first gather requirements across service and DevOps teams to understand which visualizations are shared. Leverage Datadog's OOTB dashboards and [Powerpacks](#) in order to decrease time to value, then build any other shared dashboard components into groups. While custom dashboards do not incur data storage costs, using Powerpacks and/or creating standard dashboard groups can reduce duplicative work across the team, avoiding additional operating costs to build out dashboards.

Next, keeping the intended audience in mind, consider the balance between too many dashboards with single visualizations and too few dashboards with too much data in them. It is helpful to require justifications for data use. The following sample list of questions may not apply to every dashboard metric, but it does offer guidance against reporting on more data than is useful.

Sample justification questions

- Who will be using this metric?
- Why this metric?
- How will this data be used?
- How often?
- Will this be shared with others?
- In the absence of this metric, who would be impacted?

Sample list of dashboard audiences

- Systems administrators
- Application owners and developers
- Managers and executives
- Eyes-on-glass operators

Each audience has its own specific requirements and may need different metrics. The latest OOTB dashboard content can be found in the Datadog [release notes](#) (app login required).

Dashboards are often the foundation of reporting, but not all dashboards should be turned into recurring reports. Reports require additional work to develop and maintain, so they should be kept to a minimum to reduce maintenance costs. During upgrades or major clean-up projects, it is often discovered that a long list of reports are being maintained despite a lack of evidence that anyone is viewing them. As a rule of thumb, reports should be created when the primary users will be viewing the data outside of Datadog or extended retention periods are desired.

Key Benefits

- Improved Uptime/Availability through standard dashboards and SLOs tied to business SLIs (minimizing resources dedicated to addressing incidents and avoiding customer impacts)
- Minimizing dashboard bloat/sprawl, thereby avoiding additional resource costs to maintain a greater number of dashboards/reports and elongating the time to find a critical metric(s) across dashboards

SLIs and SLOs

It is a best practice to establish SLOs across teams in order to increase and track product reliability. This [Datadog blog series on the fundamentals of SLOs](#) walks through the thinking in more detail; in a nutshell, defining SLOs ties individual availability metrics to customer experience. SLOs should be aggregated into one dashboard for management, but should also be included in dashboards for application developers and eyes-on-glass operators for transparency. The resulting increase in accountability for service uptime can then lead to realized improvement in customer experience.

Key Benefits:

- Improves uptime/availability through standard dashboards and SLOs tied to business SLIs, minimizing resources dedicated to addressing incidents and avoiding customer impact
- Minimizes dashboard bloat/sprawl, reducing the resource costs required to maintain a greater number of dashboards and reports, as well as the time to find critical metrics across dashboards

Play 3

Alerts Requiring Immediate Action

The first level of monitoring considers issues that require immediate action via alerting. This category of alerts relates to issues with a critical business impact, and should be exclusively a high enough priority to warrant waking someone up in the middle of the night. Monitors for these types of alerts should use Datadog's [alerting software integrations](#).

Sample list of justification questions

- What is the purpose of this alert?
- What would be the potential business impact if this alert were not in place?
- What action(s) should be taken as a result of this alert being triggered?
- Who should be notified?
- Who is responsible for taking action?
- What is the required response time?
- What is the escalation plan if one is necessary?

For each category of issues that have a critical business impact, it is important to consider all of these questions before creating an alert as a way to determine its criticality.

As touched on previously, having a well-defined tagging strategy is vital in order to ensure a smooth rollout and the ability to maximize the platform. Datadog tags—in particular, unified service tagging—are the glue that ties all your data together. We highly recommend establishing [a tag policy](#) for all monitors in order to keep mission-critical alerts separate from other necessary alerts, as well as decrease the time to respond to and maintain them. Additionally, set up and maintain a [Service Catalog integration](#) for an on-call service in order to populate it with up-to-date information about who will be paged to ensure faster incident response.

Finally, configure alerts to include as much information in the notification as possible. Use tags, [metric template variables](#), and settings to include query content and triggering values to save time for the incident responder. Prevent teams from having to identify where to look by combining message template variables with dashboard URLs to automatically generate the correct dashboard view when an alert is triggered.

Sample list of Urgent APM monitors to configure

- Production service outage
- Production service with latency increase beyond acceptable threshold
- Production service experiencing unexpected traffic spike that can overload infrastructure
- Production service experiencing high resource utilization that can overload infrastructure
- Production service experiencing data loss
- Production service experiencing a security breach
- Production service in violation of its SLAs

Key Benefits

Reduce MTTR and FTEs required per incident by using precise, information-rich notifications, saving on resource costs during incident management

Play 4

Alerts Not Requiring Immediate Attention

Not all alerts require immediate action. Defining alerts in this category helps balance the tradeoff between mitigating an issue's milder impact to the business and operating costs for immediate resource allocation to fix the issue. Examples include forecast, informational, and warning alerts. Notifications for these types of alerts should generally be sent via email and/or messaging apps.

The first step is to verify that all immediate action alerts from Play 3 also have a warning threshold—a well-tuned warning can potentially turn a reactive incident into a proactively resolved situation, improving your KPIs.

When defining the types of issues that require alerts for non-immediate attention, avoid creating too many alerts. This can lead to alert fatigue, overwhelming notified engineers and potentially leading to increased MTTR beyond target SLOs/SLIs. While every company is different, as a very general rule of thumb, no one team should see more than five non-critical alerts per day. Alternatively, no one monitor should alert more than five times per week, as this type of data flapping indicates poor configuration.

We recommend asking the following questions for each of these alerts as a way to mitigate the risk of alert fatigue (we actually recommend asking them for all alerts, but particularly for those that will send out a notification):

Sample list of justification questions

- What is the purpose of this alert?
- What would be the potential business impact if the alert were not in place?
- What action(s) should be taken as a result of this alert being triggered?
- Who should be notified?
- Who is responsible for taking action?
- Would this be more precise if combined into a composite monitor?

Answers to these questions can be documented in the alert itself.

Sample list of additional APM monitors to configure

- Service with an abnormal change in throughput
- Service with a high error rate
- Service with a high average latency
- Service with a high p90 latency
- Service with an abnormal change in Apdex score
- Service with a faulty deployment
- Service with slow or failing dependencies

As an additional way to mitigate the risk of alert fatigue, identify monitors within your environment that are going off at a frequency that is unmanageable. These monitors should either have their thresholds set more appropriately, the recipients of the alerts updated, or be deleted if no longer useful.

The following is a sample query of monitors alerting most frequently in the last week:

The screenshot shows a query editor interface. At the top, a text input contains "Monitors with alerts last week". Below it, a dropdown menu is set to "Table". To the right, there is a toggle for "Global Time" which is turned on. The main query area shows a table with one row: "a" in the "Events" column. The search bar contains "Source:alert" and "ERROR". Below the search bar, a dropdown menu is open, showing "Show", "Count of", and "by". The "Count of" dropdown is set to "*", and the "by" dropdown is set to "Monitor ID (@monitor_id)". Below this, there is a "limit to" dropdown set to "top" and a "30" value. A "Σ" icon is also present.

Sample Query of Monitors Alerting Most Frequently in the Last Week

Once the process of defining the data and metrics required for reporting and alerting is done, the next step is to ensure that your environment is configured to collect them. It is highly recommended to use configuration management and automation tools to set and maintain the proper configurations, even for the [monitors](#) themselves.

Key Benefits

- Improve KPI performance by setting up proactive alerts and resolving them faster
- Reduce the time dedicated to managing false alerts and avoid incidents resulting from alert fatigue
- Streamline IT workflows by using the Datadog API and config management tools

4

Next Steps

An observability framework should never be considered a set-and-forget system. Reviewing the framework should be an ongoing process. As a rule of thumb, stable environments may only need to be reviewed once per year, while dynamic environments may warrant quarterly reviews. Additionally, the observability framework should be reviewed as part of every incident postmortem.

Sample list of review questions

- Has the current observability framework provided the required data to the right teams in a timely and easy-to-use manner?
- Are there any notifications that have caused confusion or dashboards where unnecessary data got in the way of more important information?
- Are there any metrics not being collected that would have reduced response time if they were?